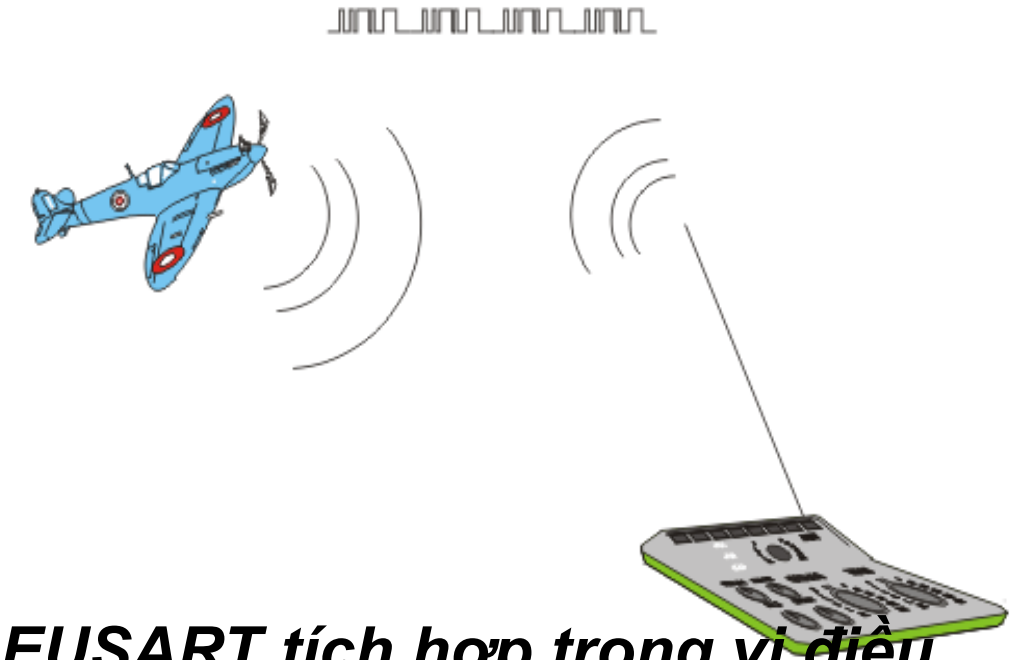
Serial Communication Modules

Instructor: Vũ Thế Đảng

1. EUSART

- **EUSART** = ngoại vi truyền thông nối tiếp của PIC16F887.
- Có thể hoạt động ở chế độ đồng bộ (synchronous) hoặc không đồng bộ (asynchronous).
- **Tích hợp sẵn:** bộ tạo xung, thanh ghi dịch, bộ đệm dữ liệu → giúp truyền/nhận dữ liệu nối tiếp tự động, không cần CPU xử lý trực tiếp.
- **Ưu điểm:** hỗ trợ truyền thông trong môi trường không có đường clock riêng, ví dụ như radio, hồng ngoại.
- Đây là giải pháp hiệu quả cho các ứng dụng truyền thông nối tiếp phức tạp.



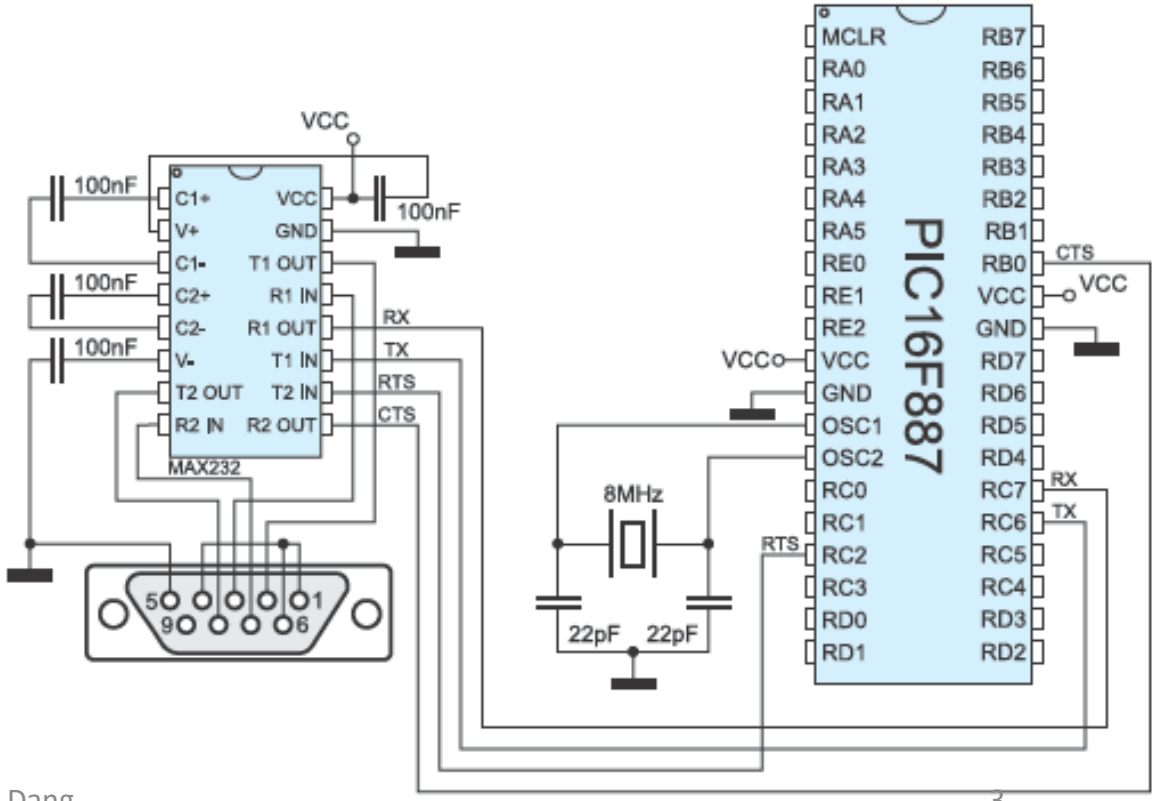
Hệ thống EUSART tích hợp trong vi điều khiển PIC16F887 có các đặc điểm sau:

- Truyền và nhận bất đồng bộ **full-duplex** (hai chiều đồng thời).
- Ký tự có độ rộng lập trình được 8 hoặc 9 bit.
- Phát hiện địa chỉ trong chế độ 9-bit.
- Phát hiện lỗi tràn bộ đệm đầu vào (input buffer overrun).
- Giao tiếp half-duplex trong chế độ đồng bộ.

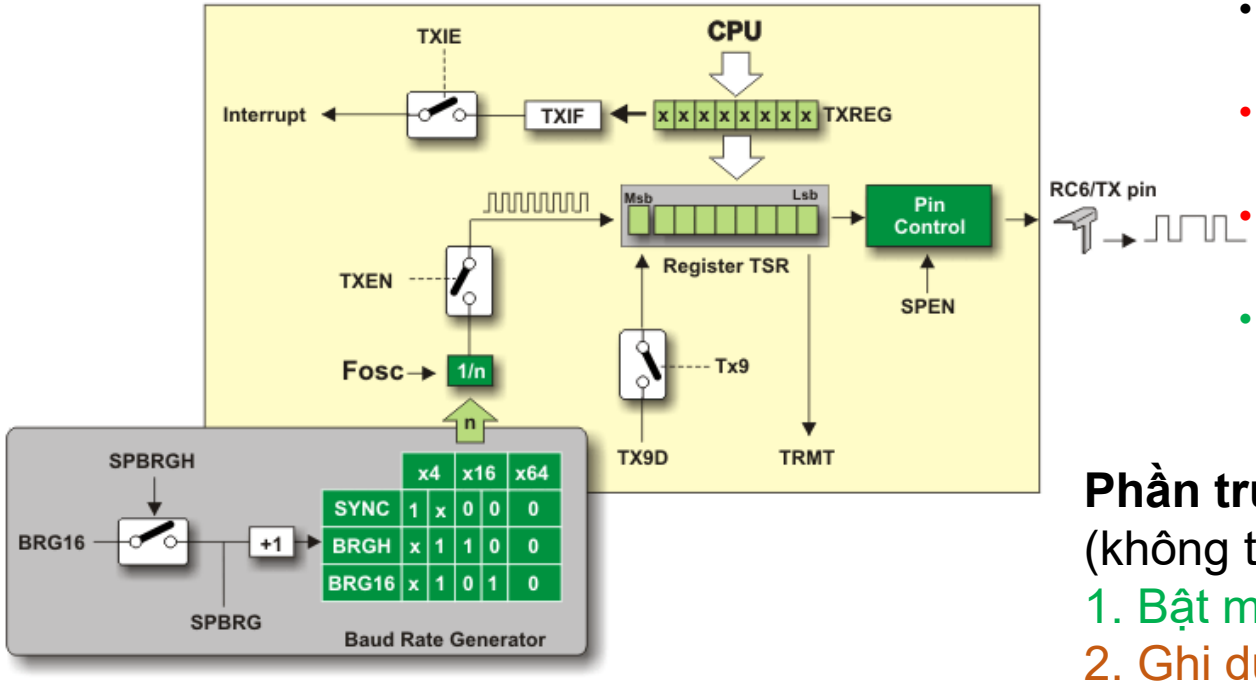
1.1. EUSART ASYNCHRONOUS MODE

Mô-đun EUSART truyền và nhận dữ liệu bằng định dạng chuẩn non-return-to-zero (NRZ). Như minh họa trong hình, chế độ này không sử dụng tín hiệu clock, trong khi định dạng dữ liệu được truyền đi lại rất đơn giản.

- **Tóm tắt ngắn gọn, mỗi dữ liệu được truyền theo cách sau:**
  - Ở trạng thái rỗi (idle state), đường dữ liệu ở mức logic cao (1).
  - Mỗi lần truyền dữ liệu bắt đầu bằng bit START, luôn là mức logic thấp (0).
  - Dữ liệu có độ rộng 8 hoặc 9 bit, trong đó bit thấp nhất (LSB) được truyền trước.
  - Mỗi lần truyền dữ liệu kết thúc bằng bit STOP, luôn là mức logic cao (1).



1.2. EUSART ASYNCHRONOUS TRANSMITTER



- **Để kích hoạt truyền dữ liệu qua mô-đun EUSART, cần cấu hình nó hoạt động ở chế độ truyền (transmitter). Cụ thể:**
- **TXEN = 1** → bật chức năng truyền EUSART bằng cách set bit TXEN trong thanh ghi TXSTA.
- SYNC = 0 → cấu hình EUSART ở chế độ bất đồng bộ bằng cách xóa bit SYNC trong TXSTA.
- **SPEN = 1** → bật EUSART bằng cách set bit SPEN trong RCSTA; lúc này chân **TX/CK tự động được cấu hình làm ngõ ra**. Nếu chân này đồng thời dùng cho chức năng analog, cần tắt bằng cách xóa bit tương ứng trong ANSEL.

Phần trung tâm của bộ truyền là thanh ghi dịch TSR (không truy cập trực tiếp được). Quy trình truyền dữ liệu:

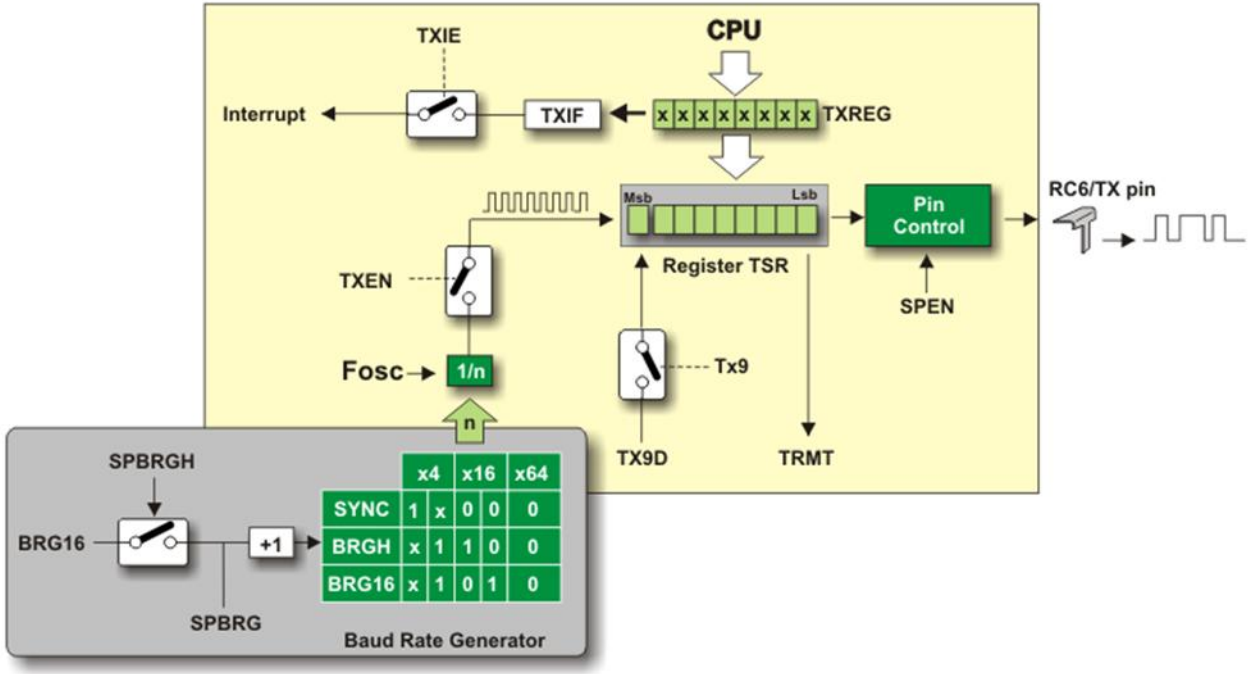
1. Bật mô-đun bằng cách set TXEN.
2. Ghi dữ liệu cần gửi vào TXREG.

• Byte sẽ ngay lập tức được chuyển sang TSR.
• TXREG trống, cờ TXIF trong PIR1 được set. Nếu TXIE trong **PIE1** được bật, sẽ phát sinh ngắt. Cờ này chỉ xóa khi ghi dữ liệu mới vào TXREG.

3. Mạch điều khiển sẽ đẩy dữ liệu ra chân TX theo đồng hồ nội bộ: START bit (0) → dữ liệu → STOP bit (1).
4. Khi bit cuối cùng rời khỏi TSR, bit TRMT trong TXSTA tự động được set.
5. Nếu TXREG đã nhận dữ liệu mới trong lúc đó, toàn bộ quá trình sẽ **lặp lại ngay sau khi bit STOP** của ký tự trước được truyền xong.
6. Truyền dữ liệu **9-bit được bật bằng cách set TX9** trong TXSTA. Bit thứ 9 (MSB) là TX9D. **Khi truyền 9-bit**, phải ghi **TX9D trước**, sau đó mới ghi 8 bit thấp vào TXREG. Tất cả 9 bit sẽ được chuyển sang TSR ngay sau khi ghi xong TXREG.

Quá trình truyền dữ liệu

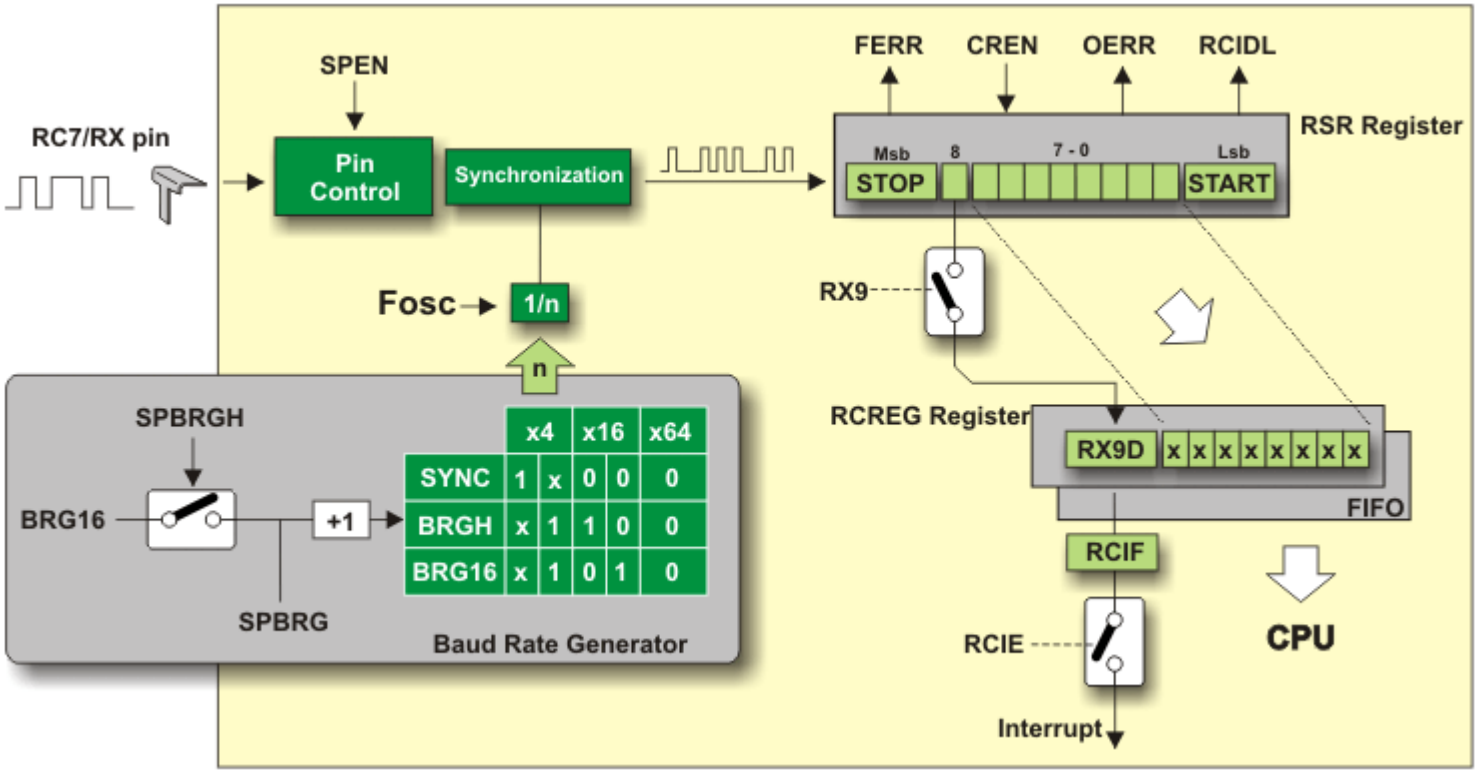
- Muốn truyền dữ liệu qua EUSART → bật TXEN, chọn chế độ bất đồng bộ (SYNC=0), bật SPEN.
- Dữ liệu ghi vào TXREG sẽ tự động chuyển sang TSR để phát đi.
- Cờ TXIF báo TXREG trống, TRMT báo TSR trống.
- Hỗ trợ ngắt truyền khi TXREG trống.
- Có thể truyền 8 hoặc 9 bit, với bit thứ 9 (TX9D) phải ghi trước.
- Quá trình truyền: START (0) → dữ liệu (LSB trước) → STOP (1).



EUSART ASYNCHRONOUS RECEIVER

Để truyền/nhận dữ liệu qua EUSART, cần cấu hình các bit điều khiển:

- CREN = 1 → bật chức năng nhận (Receiver Enable) trong thanh ghi RXSTA.
- SYNC = 0 → chọn chế độ bất đồng bộ (Asynchronous) trong thanh ghi TXSTA.
- SPEN = 1 → bật EUSART trong RCSTA, đồng thời chân RX/DT tự động thành ngõ vào. Nếu chân này đang dùng cho chức năng analog thì phải tắt trong ANSEL.



Quá trình nhận dữ liệu

1. Khi phát hiện START bit, dữ liệu được đưa vào RSR (shift register) qua chân RX.

2. Khi nhận STOP bit, dữ liệu được chuyển sang RCREG (nếu trống).

3. RCIF flag được set, và nếu bật ngắt (RCIE trong PIE1), sẽ phát sinh ngắt.

- Cờ RCIF chỉ xóa khi đọc RCREG.
- RCREG là FIFO 2 ký tự, cho phép nhận 2 byte liên tiếp.

4. Nếu RCREG đã đầy (2 byte) mà vẫn nhận thêm dữ liệu, bit OERR (Overrun Error) sẽ được set → dữ liệu mới bị mất.

- Muốn xóa OERR, phải tắt rồi bật lại CREN.
- Khi OERR còn set thì không thể nhận dữ liệu mới.

5. Nếu STOP bit = 0, bit FERR (Framing Error) trong RCSTA sẽ được set.

6. Muốn nhận dữ liệu 9-bit, phải bật bit RX9 trong RCSTA.

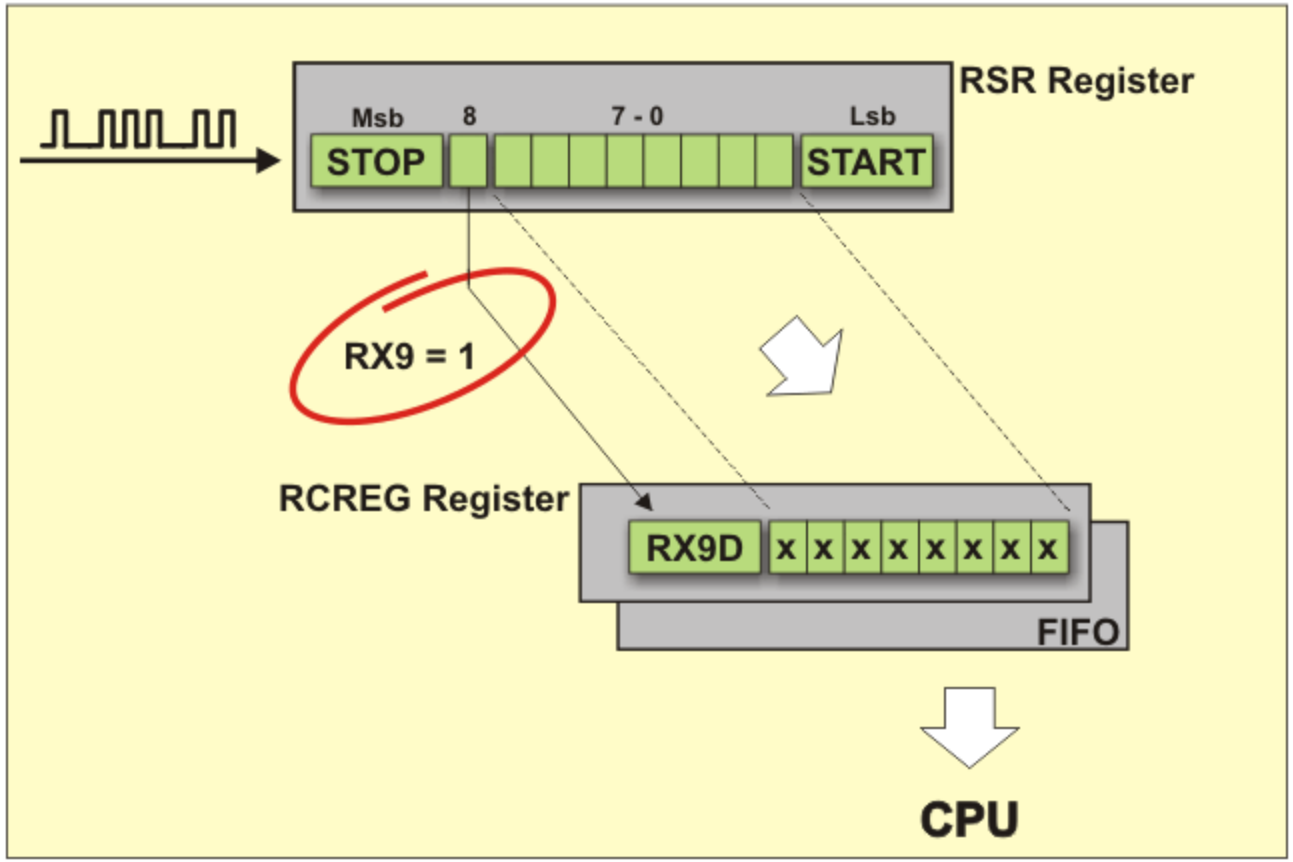
Cơ chế phát hiện lỗi khi nhận dữ liệu

- **1. Framing Error (Lỗi khung)**
- Xảy ra khi **bit STOP không được phát hiện đúng thời điểm**.
- Được báo hiệu bởi **bit FERR** trong thanh ghi **RCSTA**.
- Đặc điểm:
  - Không tự tạo ngắt.
  - Dữ liệu vừa nhận có thể sai.
  - Không ngăn cản việc nhận dữ liệu mới.
  - Bit FERR được xóa khi đọc dữ liệu nhận (phải kiểm tra trước khi đọc).
  - Không thể xóa bằng phần mềm, chỉ có thể xóa bằng cách **clear SPEN** (reset toàn bộ EUSART).

- **2. Overrun Error (Lỗi tràn bộ đệm)**
- Bộ nhớ FIFO của EUSART chỉ chứa được **2 ký tự**.
- Nếu nhận ký tự thứ 3 → **lỗi tràn**.
- Được báo hiệu bởi **bit OERR** trong thanh ghi **RCSTA**.
- Hệ quả:
  - Hai byte đã lưu trong FIFO vẫn đọc bình thường.
  - Không nhận thêm dữ liệu mới cho đến khi xóa OERR.
  - Để xóa OERR: phải **clear CREN** hoặc **clear SPEN** (reset EUSART).

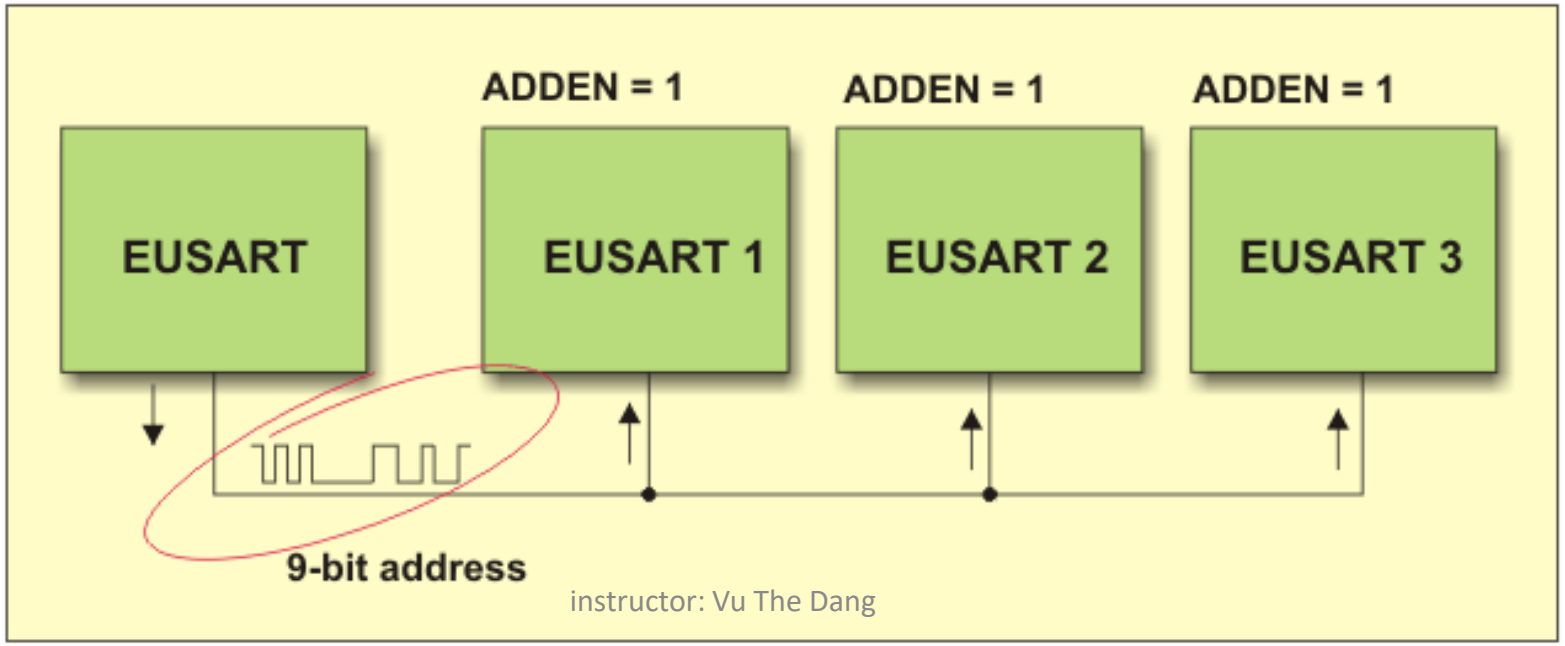
Ngoài việc nhận dữ liệu chuẩn 8-bit, hệ thống EUSART còn hỗ trợ nhận dữ liệu 9-bit.

- Ở phía truyền, bit thứ 9 được “gắn” vào byte gốc ngay trước bit STOP.
- Ở phía nhận, khi bit **RX9** trong thanh ghi **RCSTA** được set, bit dữ liệu thứ 9 sẽ tự động ghi vào **RX9D** của cùng thanh ghi.
- Sau khi nhận byte này, cần chú ý cách đọc: **bit RX9D phải được đọc trước** khi đọc 8 bit thấp trong thanh ghi **RCREG**. Nếu không, bit thứ 9 sẽ bị xóa.

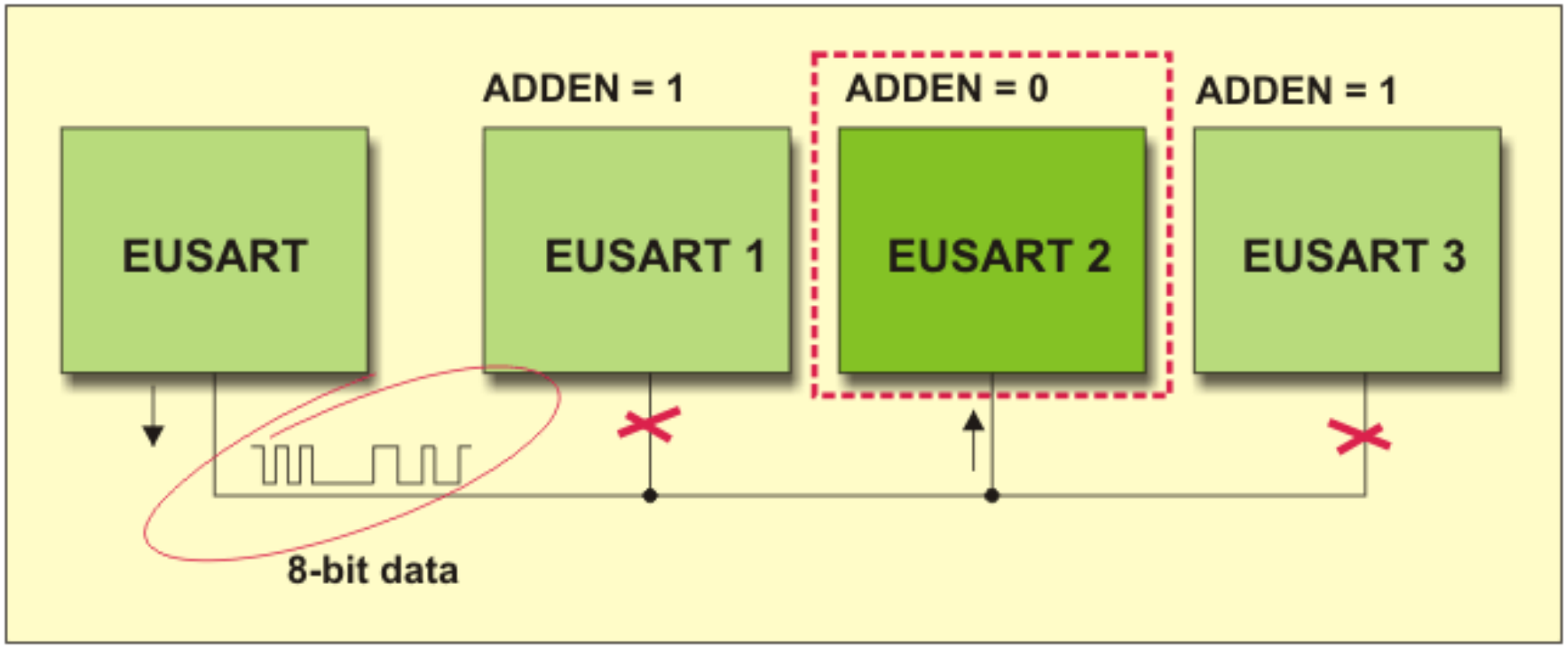


Khi bit **ADDEN** trong thanh ghi **RCSTA** được set, mô-đun EUSART chỉ nhận dữ liệu 9-bit, còn tất cả dữ liệu 8-bit sẽ bị bỏ qua. Mặc dù có vẻ như là một hạn chế, chế độ này cho phép giao tiếp nối tiếp giữa nhiều vi điều khiển. Nguyên lý hoạt động:

- Thiết bị **Master** gửi một dữ liệu 9-bit đại diện cho địa chỉ của một **Slave**.
- Tất cả các Slave trên cùng đường truyền phải bật **ADDEN** để kích hoạt chế độ phát hiện địa chỉ.
- Mỗi Slave nhận dữ liệu này và tự động kiểm tra xem có khớp với địa chỉ của mình không.
- Nếu địa chỉ khớp, phần mềm của Slave đó phải **tắt ADDEN** (clear ADDEN) để tiếp tục nhận dữ liệu sau đó.



Thiết bị Master tiếp tục gửi dữ liệu 8-bit. Tất cả dữ liệu đi qua đường truyền sẽ chỉ được nhận bởi mô-đun EUSART đã được định địa chỉ. Khi byte cuối cùng đã được nhận, thiết bị Slave cần set lại bit **ADDEN** để kích hoạt chế độ phát hiện địa chỉ mới.

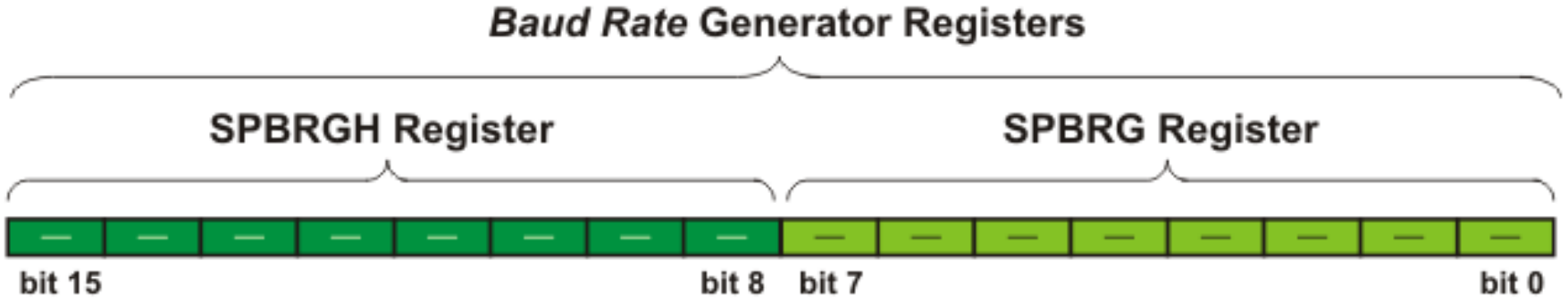


TXSTA Register

	R/W (0)	R/W (0)	R/W (0)	R/W (0)	R/W (0)	R/W (0)	R (1)	R/W (0)	Features
TXSTA	CSRC	TX9	TXEN	SYNC	SENDB	BRGH	TRMT	TX9D	Bit name
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	

•**CSRC (Clock Source Select bit)** – chọn nguồn xung clock, chỉ dùng trong chế độ **synchronous**:
•1: Master mode, clock tạo nội bộ từ Baud Rate Generator.
•0: Slave mode, clock lấy từ nguồn ngoài.
•**TX9 (9-bit Transmit Enable bit)** – cho phép truyền dữ liệu 9-bit:
•1: Truyền dữ liệu 9-bit.
•0: Truyền dữ liệu 8-bit.
•**TXEN (Transmit Enable bit)** – bật/tắt truyền:
•1: Cho phép truyền.
•0: Ngừng truyền.
•**SYNC (EUSART Mode Select bit)** – chọn chế độ hoạt động:
•1: Chế độ synchronous.
•0: Chế độ asynchronous.
•**SENDB (Send Break Character bit)** – chỉ dùng trong chế độ asynchronous, khi cần tuân thủ chuẩn LIN bus:
•1: Cho phép gửi ký tự Break.
•0: Đã hoàn tất gửi ký tự Break.
•**BRGH (High Baud Rate Select bit)** – chọn tốc độ baud trong chế độ asynchronous:
•1: Tốc độ cao.
•0: Tốc độ thấp.
•(Không ảnh hưởng trong synchronous mode).
•**TRMT (Transmit Shift Register Status bit)** – trạng thái thanh ghi dịch truyền (TSR):
•1: TSR rỗng.
•0: TSR đầy.
•**TX9D (Ninth bit of Transmit Data)** – bit thứ 9 của dữ liệu truyền, có thể dùng làm **bit địa chỉ hoặc bit parity**.

Trong sơ đồ khối của bộ thu/phát EUSART ở chế độ **asynchronous**, bạn sẽ thấy cả hai đều dùng tín hiệu clock từ **Bộ tạo tốc độ Baud – BRG** để đồng bộ. Nguồn clock này cũng được dùng trong chế độ **synchronous**.



- BRG gồm **hai thanh ghi 8-bit** kết hợp thành một thanh ghi 16-bit.
- Giá trị nạp vào hai thanh ghi này sẽ quyết định tốc độ baud.

Ngoài ra, bit **BRGH** trong thanh ghi **TXSTA** và bit **BRGH16** trong thanh ghi **BAUDCTL** cũng ảnh hưởng đến tần số clock. Công thức tính Baud Rate được cho trong bảng (tùy thuộc vào BRGH và BRGH16).

BITS			BRG / EUSART MODE	BAUD RATE FORMULA
SYNC	BRG1G	BRGH		
0	0	0	8-bit / asynchronous	Fosc / [64 (n + 1)]
0	0	1	8-bit / asynchronous	Fosc / [16 (n + 1)]
0	1	0	16-bit / asynchronous	Fosc / [16 (n + 1)]
0	1	1	16-bit / asynchronous	Fosc / [4 (n + 1)]
1	0	X	8-bit / asynchronous	Fosc / [4 (n + 1)]
1	1	X	16-bit / asynchronous	Fosc / [4 (n + 1)]

Các bảng ở phần sau chứa giá trị cần ghi vào thanh ghi 16-bit SPBRG và cấu hình các bit SYNC, BRGH, BRGH16 để đạt được các tốc độ baud chuẩn.Để xác định tốc độ baud, sử dụng các công thức sau (tùy thuộc chế độ và cấu hình bit):

$$\text{Desired Baud Rate} = \frac{Fosc}{64(SPBRGH:SPBRG - 1)}$$

$$SPBRGH:SPBRG = \frac{\frac{Fosc}{\text{Desired Baud Rate}}}{64}$$

$$\text{Error [\%]} = \frac{\text{Calc.Baud Rate - Desired Baud Rate}}{\text{Desired Baud Rate}}$$

Baud Rate	SYNC = 0, BRGH = 0, BRG16 = 0											
	Fosc = 20 MHz			Fosc = 18.432 MHz			Fosc = 11.0592 MHz			Fosc = 11.0592 MHz		
	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)
300	-	-	-	-	-	-	-	-	-	-	-	-
1200	1221	1.73	255	1200	0.00	239	1200	0.00	143	1202	0.16	103
2400	2404	0.16	129	2400	0.00	119	2400	0.00	71	2404	0.16	51
9600	9470	-1.36	32	9600	0.00	29	9600	0.00	17	9615	0.16	12
10417	10417	0.00	29	10286	-1.26	27	10165	-2.42	16	10417	0.00	11
19.2k	19.53	1.73	15	19.2	0.00	14	19.2	0.00	8	-	-	-
57.6k	-	-	-	57.6k	0.00	7	57.6	0.00	2	-	-	-
115.2k	-	-	-	-	-	-	-	-	-	-	-	-

Baud Rate	SYNC = 0, BRGH = 0, BRG16 = 0											
	Fosc = 4 MHz			Fosc = 3.6864 MHz			Fosc = 2 MHz			Fosc = 1 MHz		
	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)
300	300	0.16	207	300	0.00	191	300	0.16	103	300	0.16	51
1200	1202	0.16	51	1200	0.00	47	1202	0.16	25	1202	0.16	12
2400	2404	0.16	25	2400	0.00	23	2404	0.16	12	-	-	-
9600	-	-	-	9600	0.00	5	-	-	-	-	-	-
10417	10417	0.00	5	-	-	-	10417	0.00	2	-	-	-
19.2k	-	-	-	19.2	0.00	2	-	-	-	-	-	-
57.6k	-	-	-	57.6k	0.00	0	-	-	-	-	-	-
115.2k	-	-	-	-	-	-	-	-	-	-	-	-

Baud Rate	SYNC = 0, BRGH = 1, BRG16 = 0											
	Fosc = 20 MHz			Fosc = 18.432 MHz			Fosc = 11.0592 MHz			Fosc = 8 MHz		
	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)
300	-	-	-	-	-	-	-	-	-	-	-	-
1200	-	-	-	-	-	-	-	-	-	-	-	-
2400	-	-	-	-	-	-	-	-	-	2404	0.16	207
9600	9615	0.16	129	9600	0.00	119	9600	0.00	71	9615	0.16	51
10417	10417	0.00	119	10378	-0.37	110	10473	0.53	65	10417	0.00	47
19.2k	19.23k	0.16	64	19.2	0.00	59	19.2k	0.00	35	19231	0.16	25
57.6k	56.82k	-1.36	21	57.6k	0.00	19	57.6k	0.00	11	55556	-3.55	8
115.2k	113.64k	-1.36	10	115.2k	0.00	9	115.2k	0.00	5	-	-	-

BAUDCTL Register

	R (0)	R (1)		R/W (0)	R/W (0)		R/W (0)	R/W (0)	Features
BAUDCTL	ABDOVF	RCIDL	-	SCKP	BRG16	-	WUE	ABDEN	Bit name
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	

- **ABDOVF (Auto-Baud Detect Overflow bit)** – chỉ dùng trong chế độ **asynchronous** khi phát hiện tốc độ baud:
- 1: Bộ định thời auto-baud bị tràn.
- 0: Bộ định thời chưa tràn.
- **RCIDL (Receive Idle Flag bit)** – chỉ dùng trong chế độ **asynchronous**:
- 1: Bộ thu đang rảnh (idle).
- 0: Đã nhận START bit, quá trình nhận dữ liệu đang diễn ra.
- **SCKP (Synchronous Clock Polarity Select bit)** – chọn cực tính đồng hồ, tùy chế độ:
- **Asynchronous**:
  - 1: Dữ liệu truyền ra chân RC6/TX/CK bị đảo.
  - 0: Dữ liệu truyền không đảo.
- **Synchronous**:
  - 1: Đồng bộ theo cạnh lên của xung clock.
  - 0: Đồng bộ theo cạnh xuống của xung clock.
- **BRG16 (16-bit Baud Rate Generator bit)** – chọn độ dài bộ tạo tốc độ baud:
- 1: Dùng bộ tạo baud 16-bit (SPBRGH + SPBRG).
- 0: Dùng bộ tạo baud 8-bit (SPBRG).
- **WUE (Wake-up Enable bit)** – cho phép đánh thức vi điều khiển từ chế độ ngủ:
- 1: Bộ thu chờ cạnh xuống trên chân RC7/RX/DT để đánh thức MCU.
- 0: Bộ thu hoạt động bình thường.
- **ABDEN (Auto-Baud Detect Enable bit)** – chỉ dùng trong chế độ **asynchronous**:
- 1: Bật chế độ tự động phát hiện baud. Bit này sẽ tự động được xóa sau khi phát hiện thành công.
- 0: Tắt chế độ tự động phát hiện baud.

MASTER SYNCHRONOUS SERIAL PORT MODULE (MSSP)

MSSP là một trong những mạch phức tạp nhưng rất hữu ích của vi điều khiển. Nó cho phép giao tiếp tốc độ cao giữa vi điều khiển và các ngoại vi hoặc vi điều khiển khác chỉ với 2–3 chân I/O. Vì vậy, MSSP thường dùng để kết nối với LCD, bộ chuyển đổi A/D, EEPROM nối tiếp, thanh ghi dịch, v.v.

Đặc điểm chính:

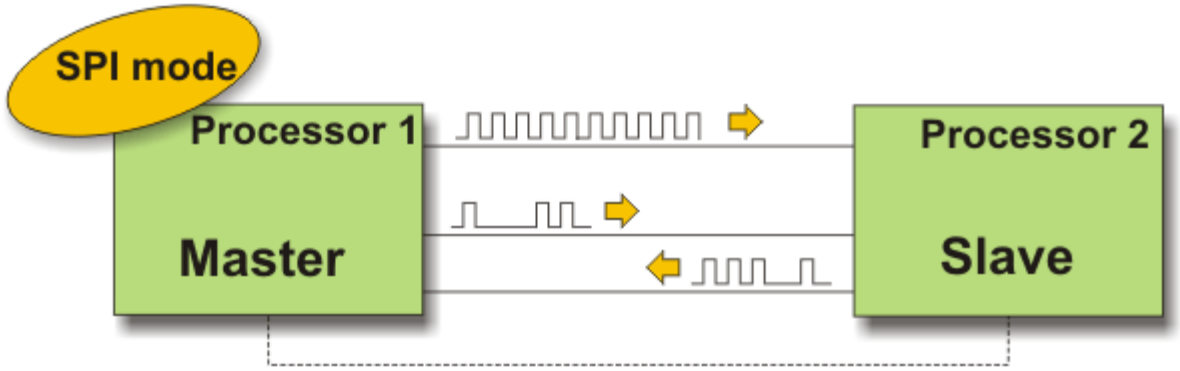
- **Giao tiếp đồng bộ (synchronous)**, phù hợp hệ thống có một Master và nhiều Slave.
- Thiết bị **Master** có mạch tạo clock (baud rate generator) và cung cấp xung clock cho toàn hệ thống.
- Thiết bị **Slave** không cần mạch tạo clock nội bộ.

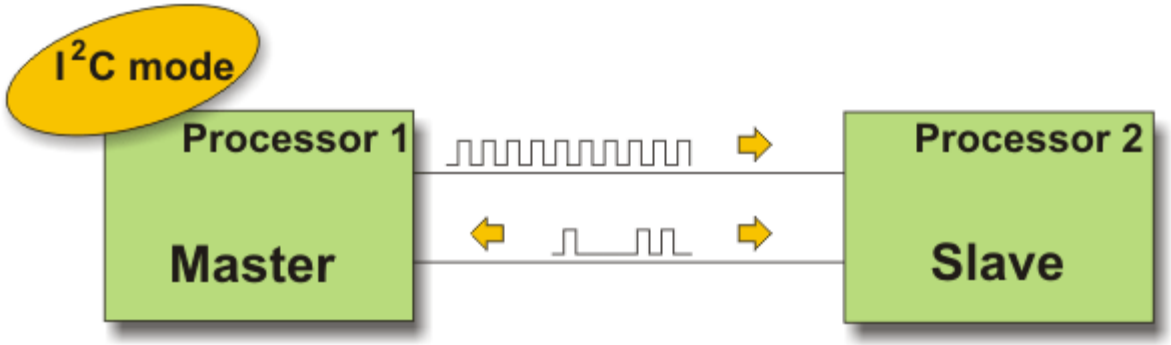
MSSP có thể hoạt động ở hai chế độ:

- **SPI (Serial Peripheral Interface)**
- **I^2C (Inter-Integrated Circuit)**

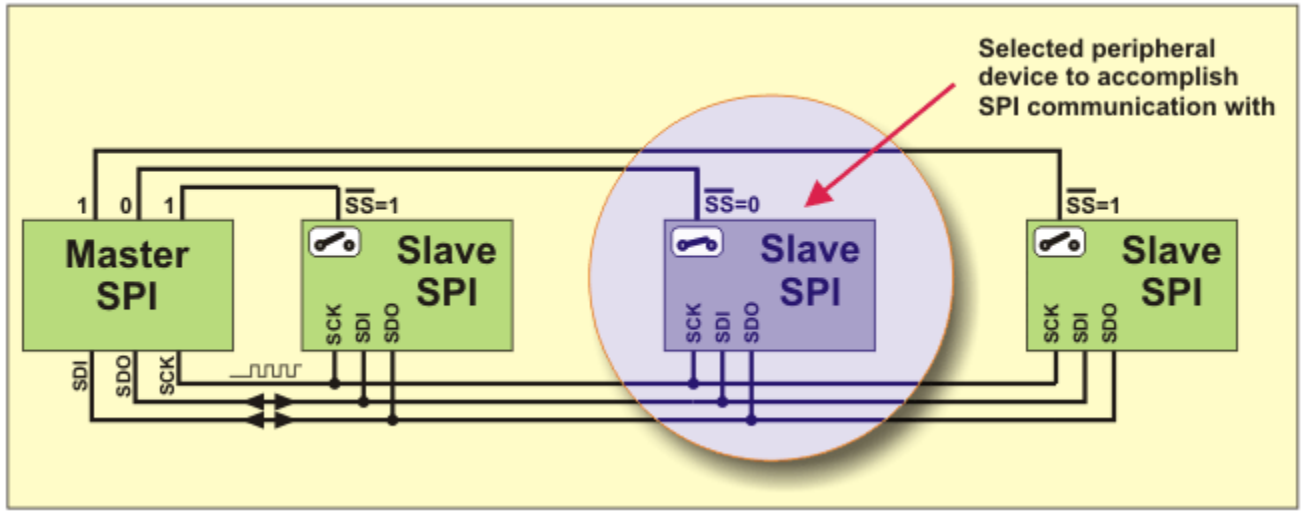
Một MSSP chỉ là một nửa phần cứng cần thiết để giao tiếp nối tiếp; nửa còn lại nằm ở thiết bị đối tác. Dù phần cứng giống nhau, chế độ hoạt động khác nhau tùy thuộc vào việc thiết bị đóng vai trò **Master** hay **Slave**.

- Nếu vi điều khiển điều khiển ngoại vi → nó phải là **Master**.
- Master sẽ tạo clock khi cần, tức là chỉ khi phần mềm yêu cầu truyền/nhận dữ liệu.
- Việc thiết lập kết nối phụ thuộc hoàn toàn vào Master.
- Ngược lại, nếu vi điều khiển được lập trình nằm trong một thiết bị phức tạp hơn (ví dụ như máy tính PC), thì nó phải hoạt động ở chế độ **Slave**. Trong vai trò này, nó luôn phải chờ yêu cầu truyền dữ liệu từ thiết bị **Master**.





SPI MODE



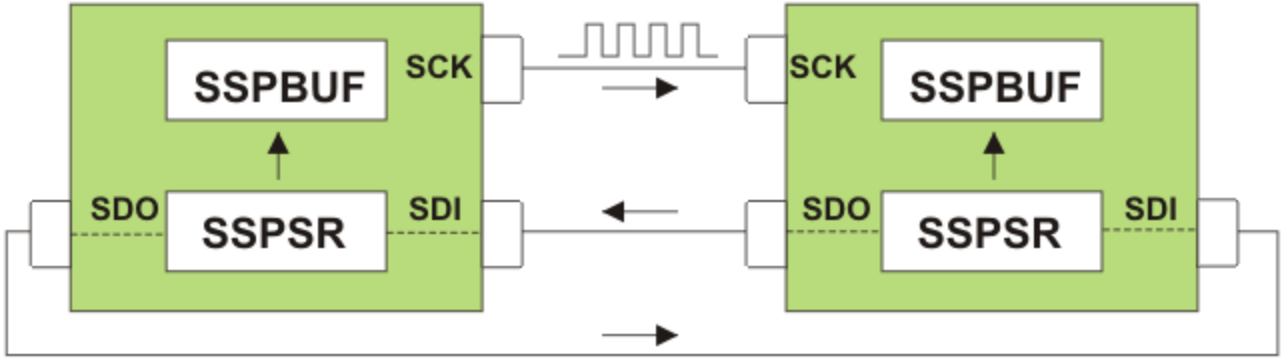
- Chế độ **SPI** cho phép truyền và nhận đồng thời 8 bit dữ liệu bằng 3 chân I/O:
- **SDO (Serial Data Out)** – đường truyền dữ liệu ra.
- **SDI (Serial Data In)** – đường nhận dữ liệu vào.
- **SCK (Serial Clock)** – đường đồng bộ clock.
- Ngoài ra còn có chân thứ tư:
- **SS (Slave Select)** – dùng khi vi điều khiển giao tiếp với nhiều ngoại vi. Chân này chỉ hoạt động khi vi điều khiển ở chế độ **Slave**, tức là khi thiết bị Master yêu cầu trao đổi dữ liệu.
- Trong chế độ SPI, MSSP sử dụng tổng cộng 4 thanh ghi:
- **SSPSTAT** – thanh ghi trạng thái.
- **SSPCON** – thanh ghi điều khiển.
- **SSPBUF** – thanh ghi đệm.
- **SSPSR** – thanh ghi dịch (không truy cập trực tiếp, dùng để chuyển đổi dữ liệu sang dạng nối tiếp).
- Ba thanh ghi đầu có thể đọc/ghi bất kỳ lúc nào, còn thanh ghi dịch chỉ hoạt động nội bộ.

Trong sơ đồ khối của **SPI module**, phần trung tâm chính là **hai thanh ghi** được kết nối trực tiếp với các chân truyền, nhận và đồng bộ:

- Trong sơ đồ khối của **SPI module**, phần trung tâm chính là **hai thanh ghi** được kết nối trực tiếp với các chân truyền, nhận và đồng bộ:
- **SSPSR (Shift Register)**: thanh ghi dịch, thực hiện việc dịch tuần tự dữ liệu theo xung clock. Đây là nơi dữ liệu song song được chuyển thành dữ liệu nối tiếp (khi truyền) hoặc ngược lại (khi nhận). Thanh ghi này không thể truy cập trực tiếp từ phần mềm.
- **SSPBUF (Buffer Register)**: thanh ghi đệm, nơi phần mềm đọc dữ liệu nhận được hoặc ghi dữ liệu cần truyền. Khi ghi vào SSPBUF, dữ liệu sẽ tự động được chuyển sang SSPSR để dịch tuần tự ra chân SDO. Khi nhận, dữ liệu từ SSPSR sẽ được chuyển vào SSPBUF để phần mềm đọc.

Các thanh ghi này liên kết với các chân:

- SDO (Serial Data Out) – dữ liệu truyền ra.
- SDI (Serial Data In) – dữ liệu nhận vào.
- SCK (Serial Clock) – xung đồng bộ do Master cung cấp.
- SS (Slave Select) – chọn thiết bị Slave khi có nhiều ngoại vi.

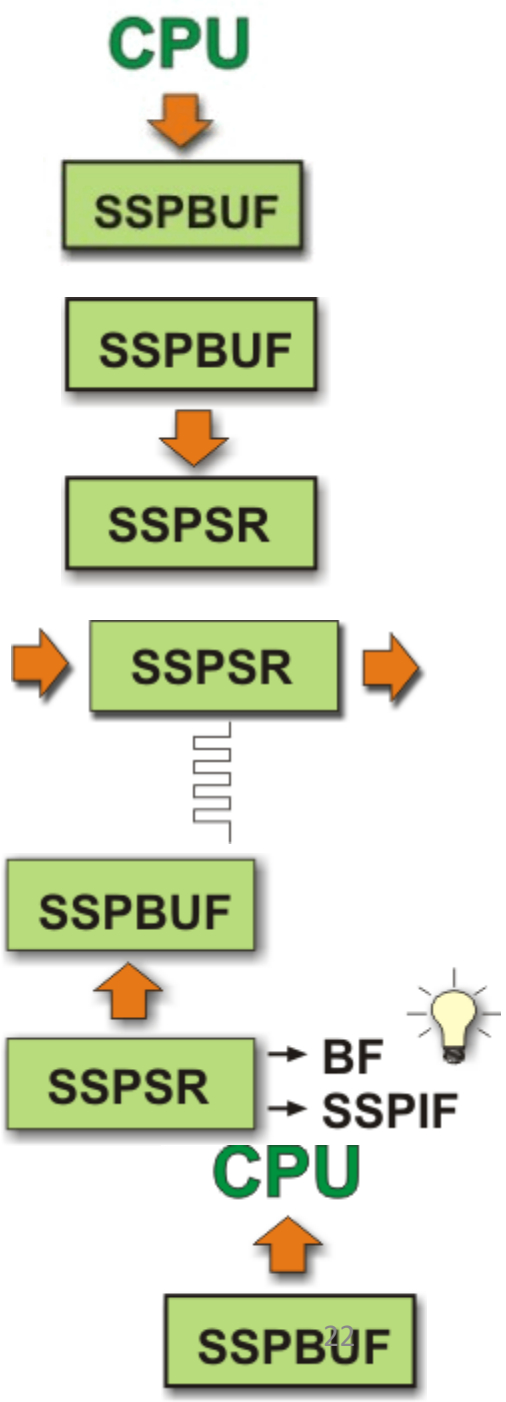


Trước khi khởi tạo SPI, cần cấu hình các tùy chọn:

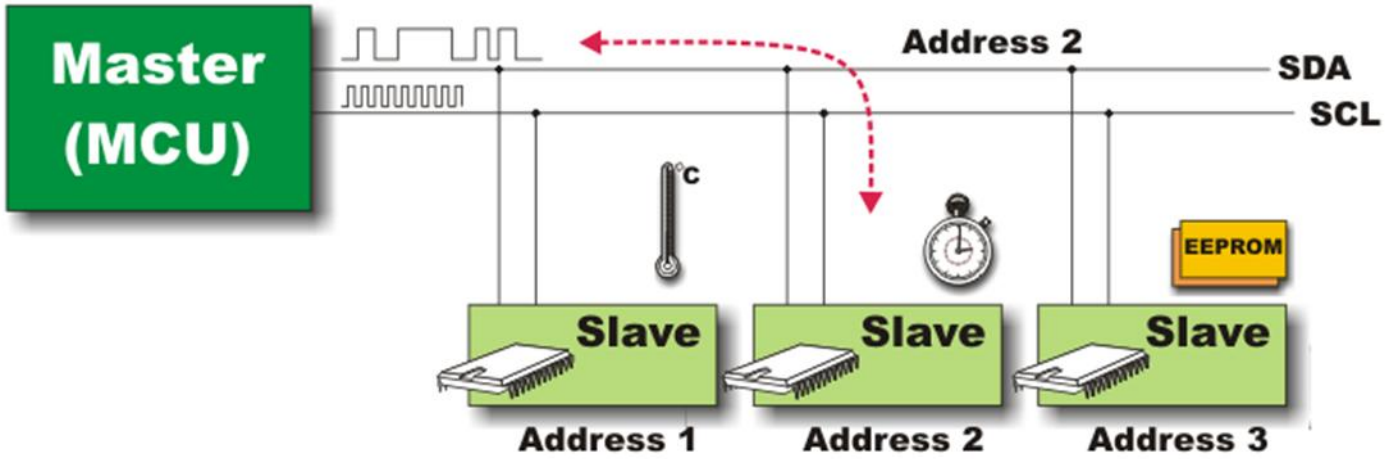
- **Master mode**: TRISC.3 = 0 (chân SCK là clock output).
- **Slave mode**: TRISC.3 = 1 (chân SCK là clock input).
- **Pha lấy mẫu dữ liệu**: giữa hoặc cuối thời gian xuất dữ liệu (bit **SMP** trong thanh ghi **SSPSTAT**).
- **Cạnh clock**: bit **CKE** trong **SSPSTAT**.
- **Tốc độ Baud**: các bit **SSPM3–SSPM0** trong **SSPCON** (chỉ Master).
- **Chế độ chọn Slave**: các bit **SSPM3–SSPM0** trong **SSPCON** (chỉ Slave).
- Bật hoạt động bằng cách set bit **SSPEN**.

Quy trình truyền/nhận dữ liệu SPI:

1. Ghi dữ liệu cần truyền vào thanh ghi **SSPBUF**.
   - Nếu ở Master mode → vi điều khiển tự động thực hiện các bước tiếp theo.
   - Nếu ở Slave mode → chỉ thực hiện khi có tín hiệu clock trên chân SCK.
2. Dữ liệu được chuyển sang thanh ghi dịch **SSPSR**, trong khi **SSPBUF** vẫn giữ nguyên.
3. Dữ liệu được dịch ra chân **SDO** (MSB trước), đồng thời nhận dữ liệu vào qua chân **SDI**.
   - Master tự tạo clock.
   - Slave dùng clock ngoài từ chân SCK.
4. Khi **SSPSR** nhận đủ 8 bit, bit **BF** trong **SSPSTAT** và bit **SSPIF** trong **PIR1** được set.
   - Dữ liệu nhận được tự động chuyển từ **SSPSR** sang **SSPBUF**.
   - Quá trình truyền diễn ra tự động, chương trình chính vẫn chạy bình thường.
   - Bit **SSPIF** có thể dùng để tạo ngắt khi truyền xong 1 byte.
5. Cuối cùng, dữ liệu trong **SSPBUF** sẵn sàng để đọc và chuyển đến thanh ghi mong muốn.

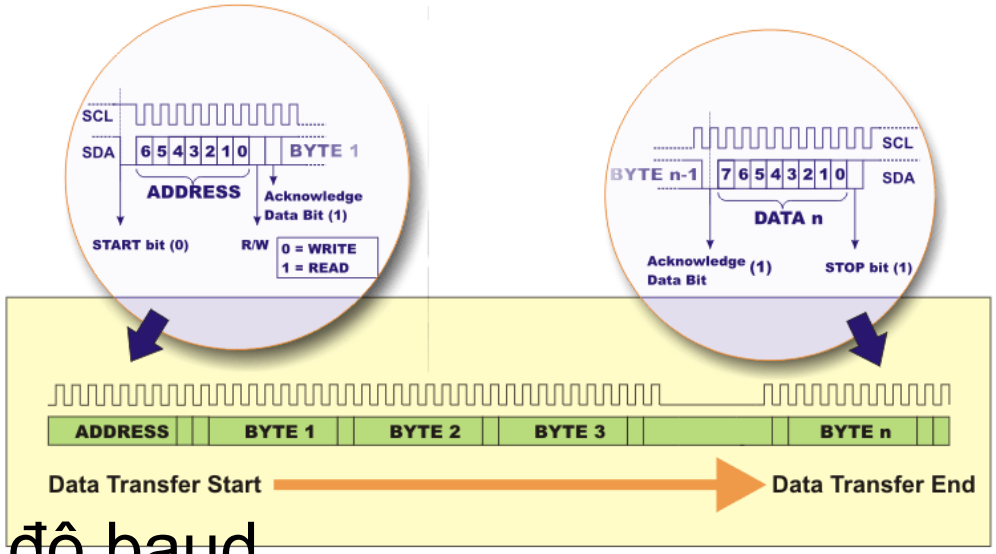


I²C MODE



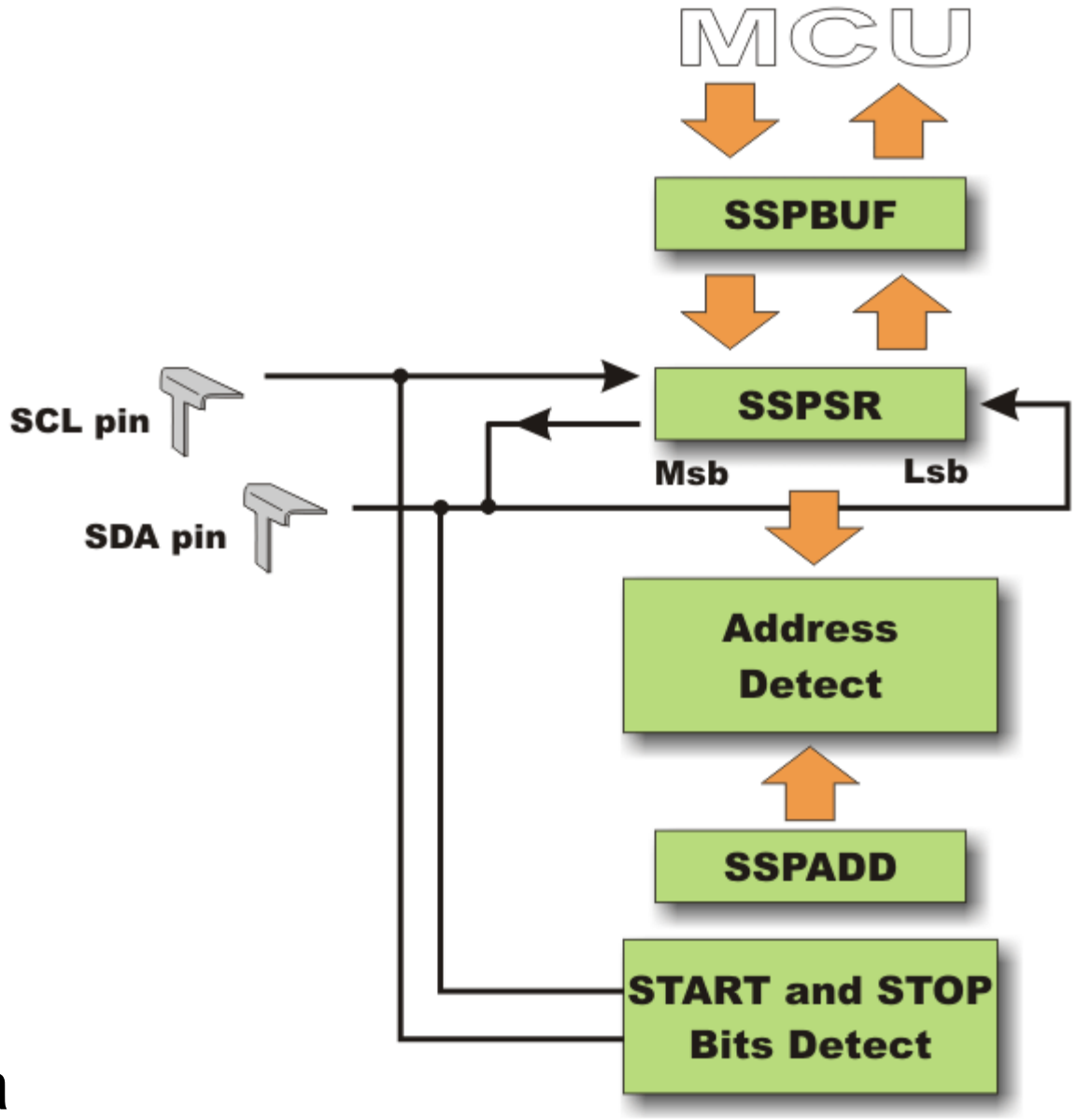
- Chế độ **I²C (Inter-IC Bus)** đặc biệt phù hợp khi vi điều khiển và IC ngoại vi nằm trong cùng thiết bị. Thường là các vi điều khiển khác hoặc các IC chuyên dụng giá rẻ thuộc thế hệ "smart peripheral" như bộ nhớ, cảm biến nhiệt độ, đồng hồ thời gian thực (RTC)…
- Giống SPI, I²C là giao tiếp **đồng bộ, hai chiều**.
- Chỉ dùng **2 chân**:
  - **SDA (Serial Data)** – dữ liệu nối tiếp.
  - **SCL (Serial Clock)** – xung clock nối tiếp.
- Người dùng phải cấu hình SDA/SCL là input/output qua các bit **TRISC**.
- Theo giao thức I²C, có thể kết nối tới **122 thiết bị** trên cùng bus chỉ với 2 chân I/O

Nguyên lý hoạt động:



- Clock luôn do **Master** tạo, tần số quyết định tốc độ baud.
- Chuẩn phổ biến: **100 kHz** (chuẩn cao tốc 3.4 MHz ít dùng).
- Mọi trao đổi dữ liệu đều do Master khởi tạo.
- Khi MSSP được bật, nó chờ **Start condition**:
  - Master gửi **START bit** (logic 0) qua SDA.
  - Sau đó gửi **7-bit địa chỉ** của Slave.
  - Cuối cùng gửi bit R/W (0 = ghi, 1 = đọc).
- Tất cả Slave trên bus nhận byte đầu tiên, nhưng chỉ Slave có địa chỉ khớp mới phản hồi.
- Sau khi gửi byte đầu tiên, Master chờ **ACK** từ Slave. Nếu Slave gửi ACK, truyền dữ liệu tiếp tục cho đến khi Master gửi **STOP bit**.

- Ngoài ra, I²C còn hỗ trợ địa chỉ 10-bit (tối đa 1024 thiết bị, nhiều Master), nhưng ít dùng thực tế.
- **Các thanh ghi MSSP dùng trong I²C:**
- **SSPCON** – điều khiển.
- **SSPCON2** – điều khiển bổ sung.
- **SSPSTAT** – trạng thái.
- **SSPBUF** – đệm dữ liệu.
- **SSPSR** – thanh ghi dịch (không truy cập trực tiếp).
- **SSPADD** – chứa địa chỉ Slave hoặc giá trị tốc độ baud.



SSPSTAT Register

- **SMP (Sample bit) SPI Master**: xác định pha lấy mẫu dữ liệu.
  - 1: đọc logic ở cuối thời gian xuất dữ liệu.
  - 0: đọc logic ở giữa thời gian xuất dữ liệu.
- **SPI Slave**: bit này phải được xóa. **I²C**:
  - 1: tắt điều khiển slew rate (chuẩn tốc độ thường 100 kHz).
  - 0: bật điều khiển slew rate (chuẩn tốc độ cao 400 kHz).
- **CKE (Clock Edge Select bit)** – chọn cạnh đồng bộ:
- Nếu **CKP = 0**:
  - 1: truyền dữ liệu trên cạnh lên.
  - 0: truyền dữ liệu trên cạnh xuống.
- Nếu **CKP = 1**:
  - 1: truyền dữ liệu trên cạnh xuống.
  - 0: truyền dữ liệu trên cạnh lên.
- **D/A (Data/Address bit, I²C)**
- 1: byte vừa nhận/truyền là dữ liệu.
- 0: byte vừa nhận/truyền là địa chỉ.

- **P (Stop bit, I²C)**
  - 1: STOP bit vừa được phát hiện.
  - 0: chưa phát hiện STOP bit.
- **S (Start bit, I²C)**
  - 1: START bit vừa được phát hiện.
  - 0: chưa phát hiện START bit.
- **R/W (Read/Write bit, I²C)**
- **Slave mode**:
  - 1: đọc dữ liệu.
  - 0: ghi dữ liệu.
- **Master mode**:
  - 1: đang truyền.
  - 0: không truyền.
- **UA (Update Address bit, I²C 10-bit)**
  - 1: cần cập nhật SSPADD.
  - 0: địa chỉ trong SSPADD đúng, không cần cập nhật.
- **BF (Buffer Full Status bit) SPI/I²C nhận dữ liệu**:
  - 1: nhận xong, SSPBUF đầy.
  - 0: chưa nhận xong, SSPBUF rỗng.
- **I²C truyền dữ liệu**:
  - 1: đang truyền dữ liệu (không tính ACK/STOP).
  - 0: truyền xong dữ liệu (không tính ACK/STOP).

	R/W (0)	R/W (0)	R (0)	R (0)	R (0)	R (0)	R (0)	R (0)	Features
SSPSTAT	SMP	CKE	D/A	P	S	R/W	UA	BF	Bit name
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	

SSPCON Register

SSPCON	R/W (0)	R/W (0)	R/W (0)	R/W (0)	R/W (0)	R/W (0)	R/W (0)	R/W (0)	Features
	WCOL	SSPOV	SSPEN	CKP	SSPM3	SSPM2	SSPM1	SSPM0	Bit name
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	

- **WCOL (Write Collision Detect bit)**
- 1: Phát hiện va chạm – ghi vào SSPBUF khi điều kiện I²C chưa hợp lệ để bắt đầu truyền.
- 0: Không có va chạm.
- **SSPOV (Receive Overflow Indicator bit)**
- 1: Nhận byte mới trước khi đọc byte cũ → tràn dữ liệu, byte trong SSPSR bị mất.
- 0: Dữ liệu nối tiếp được nhận đúng.
- **SSPEN (Synchronous Serial Port Enable bit)** – bật/tắt MSSP và cấu hình chân:
- **SPI mode**:
  - 1: Bật MSSP, cấu hình SCK, SDO, SDI, SS làm chân giao tiếp nối tiếp.
  - 0: Tắt MSSP, các chân này trở thành I/O bình thường.
- **I²C mode**:
  - 1: Bật MSSP, cấu hình SDA, SCL làm chân giao tiếp nối tiếp.
  - 0: Tắt MSSP, các chân này trở thành I/O bình thường.
- **CKP (Clock Polarity Select bit)**
- **SPI mode**:
  - 1: Clock ở trạng thái nghỉ mức cao.
  - 0: Clock ở trạng thái nghỉ mức thấp.
- **I²C slave mode**:
  - 1: Cho phép clock.
  - 0: Giữ clock ở mức thấp (dùng để tạo thêm thời gian ổn định dữ liệu).
- (Không dùng trong I²C master mode).

SSPM3-SSPM0 - Synchronous Serial Port Mode Select bits. SSP mode is determined by combining these bits:

SSPM3	SSPM2	SSPM1	SSPM0	MODE
0	0	0	0	SPI master mode, clock = Fosc/4
0	0	0	1	SPI master mode, clock = Fosc/16
0	0	1	0	SPI master mode, clock = Fosc/64
0	0	1	1	SPI master mode, clock = (output TMR)/2
0	1	0	0	SPI slave mode, SS pin control enabled
0	1	0	1	SPI slave mode, SS pin control disabled, SS can be used as I/O pin
0	1	1	0	I^2C slave mode, 7-bit address used
0	1	1	1	I^2C slave mode, 10-bit address used
1	0	0	0	I^2C master mode, clock = Fosc / [4(SSPAD+1)]
1	0	0	1	Mask used in I^2C slave mode
1	0	1	0	Not used
1	0	1	1	I^2C controlled master mode
1	1	0	0	Not used
1	1	0	1	Not used
1	1	1	0	I^2C slave mode, 7-bit address used,START and STOP bits enable interrupt
1	1	1	1	I^2C slave mode, 10-bit address used,START and STOP bits enable interrupt

SSPCON2 Register

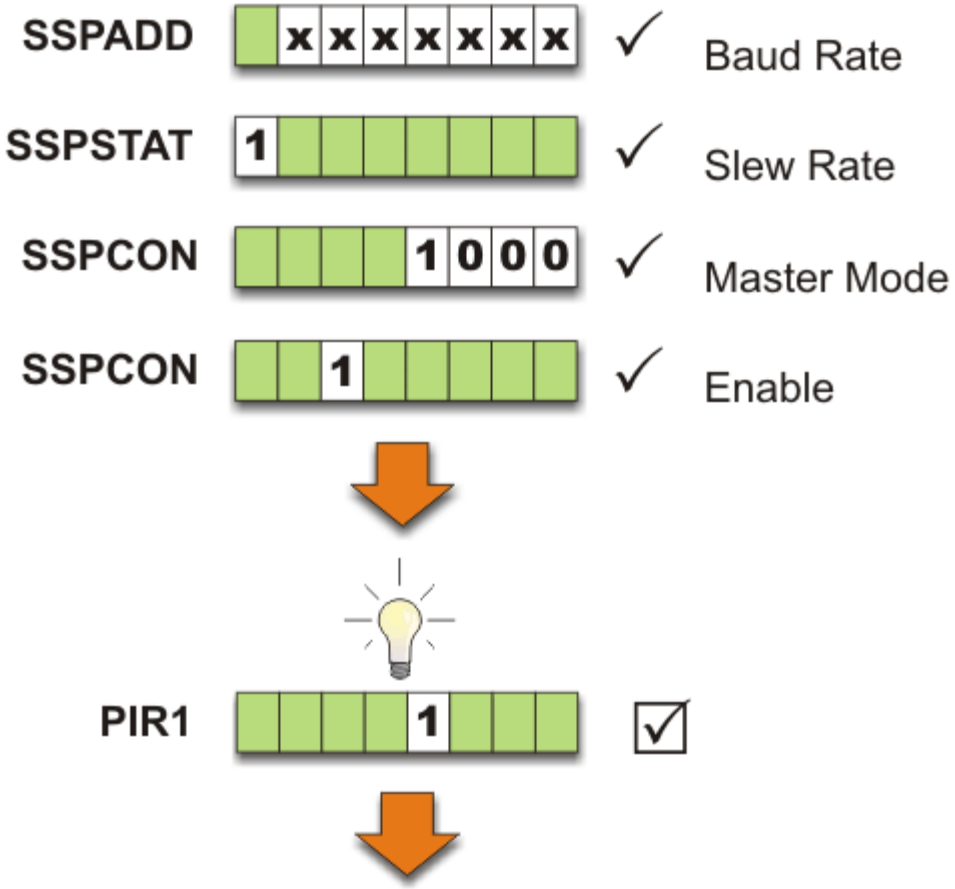
SSPCON2	R/W (0)	R (0)	R/W (0)	R/W (0)	R/W (0)	R/W (0)	R/W (0)	R/W (0)	Features
	GCEN	ACKSTAT	ACKDT	ACKEN	RCEN	PEN	RSEN	SEN	Bit name
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	

•**GCEN (General Call Enable bit, chỉ I²C Slave)**
•1: Cho phép ngắt khi nhận địa chỉ “general call” (0000h).
•0: Tắt chế độ general call.
•**ACKSTAT (Acknowledge Status bit, I²C Master Transmit)**
•1: Không nhận được ACK từ Slave.
•0: Nhận được ACK từ Slave.
•**ACKDT (Acknowledge Data bit, I²C Master Receive)**
•1: Gửi Not Acknowledge (NACK).
•0: Gửi Acknowledge (ACK).
•**ACKEN (Acknowledge Sequence Enable bit, I²C Master Receive)**
•1: Khởi tạo điều kiện ACK trên SDA/SCL và truyền bit ACKDT. Tự động xóa bởi phần cứng.
•0: Không khởi tạo ACK.

•**RCEN (Receive Enable bit, I²C Master)**
•1: Cho phép nhận dữ liệu.
•0: Tắt nhận dữ liệu.
•**PEN (STOP Condition Enable bit, I²C Master)**
•1: Khởi tạo STOP condition trên SDA/SCL. Sau đó tự động xóa.
•0: Không khởi tạo STOP.
•**RSEN (Repeated START Condition Enable bit, I²C Master)**
•1: Khởi tạo START condition lặp lại trên SDA/SCL. Sau đó tự động xóa.
•0: Không khởi tạo START lặp lại.
•**SEN (START Condition Enable bit, I²C Master)**
•1: Khởi tạo START condition trên SDA/SCL. Sau đó tự động xóa.
•0: Không khởi tạo START.

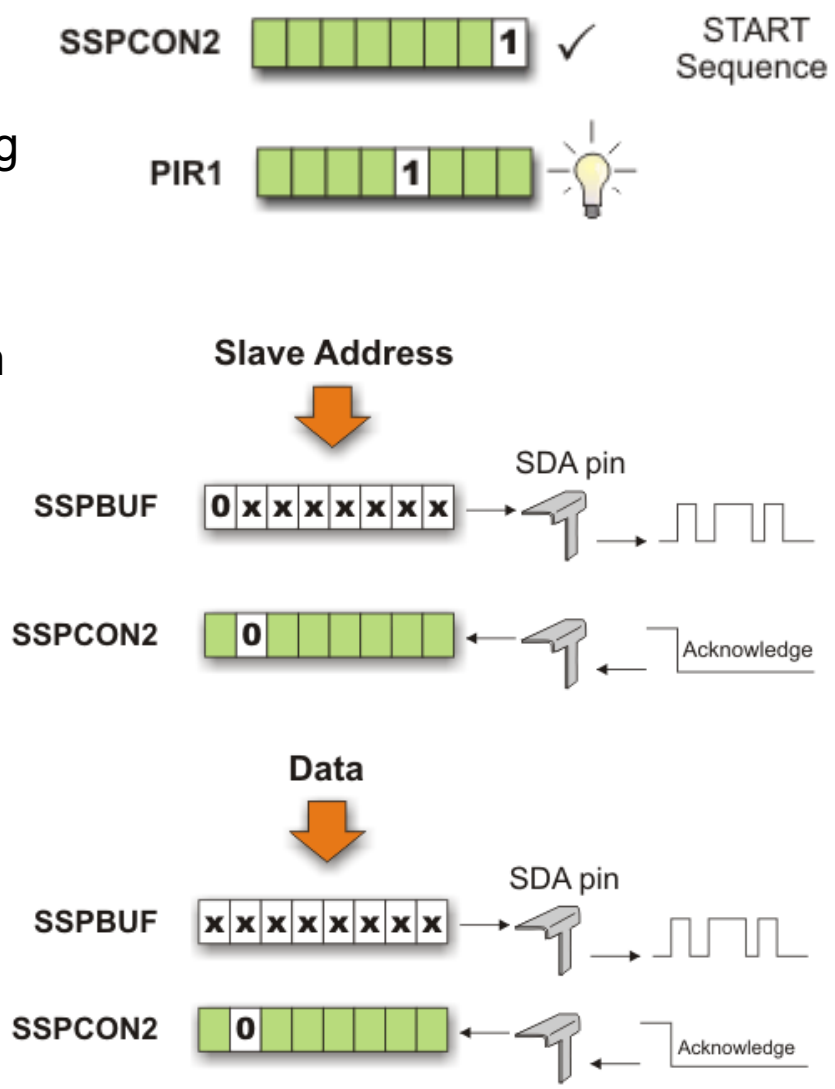
I²C in Master Mode

- Trường hợp phổ biến nhất là vi điều khiển hoạt động như **Master** và ngoại vi là **Slave**. Vì vậy, tài liệu chỉ tập trung vào chế độ này. Địa chỉ được xem là **7-bit** và hệ thống chỉ có một vi điều khiển (single-master).
- Để bật MSSP ở chế độ I²C Master, cần thực hiện:
- **Thiết lập tốc độ baud** bằng thanh ghi **SSPADD**.
- **Tắt điều khiển slew rate** bằng cách set bit **SMP** trong thanh ghi **SSPSTAT**.
- **Chọn chế độ Master** trong thanh ghi **SSPCON**.
- **Bật module MSSP** bằng cách set bit **SSPEN** trong **SSPCON**.
- Chờ phần cứng nội bộ báo sẵn sàng truyền dữ liệu → bit **SSPIF** trong thanh ghi **PIR1** được set.
- Xóa bit **SSPIF** bằng phần mềm → sau đó vi điều khiển sẵn sàng trao đổi dữ liệu với ngoại vi.



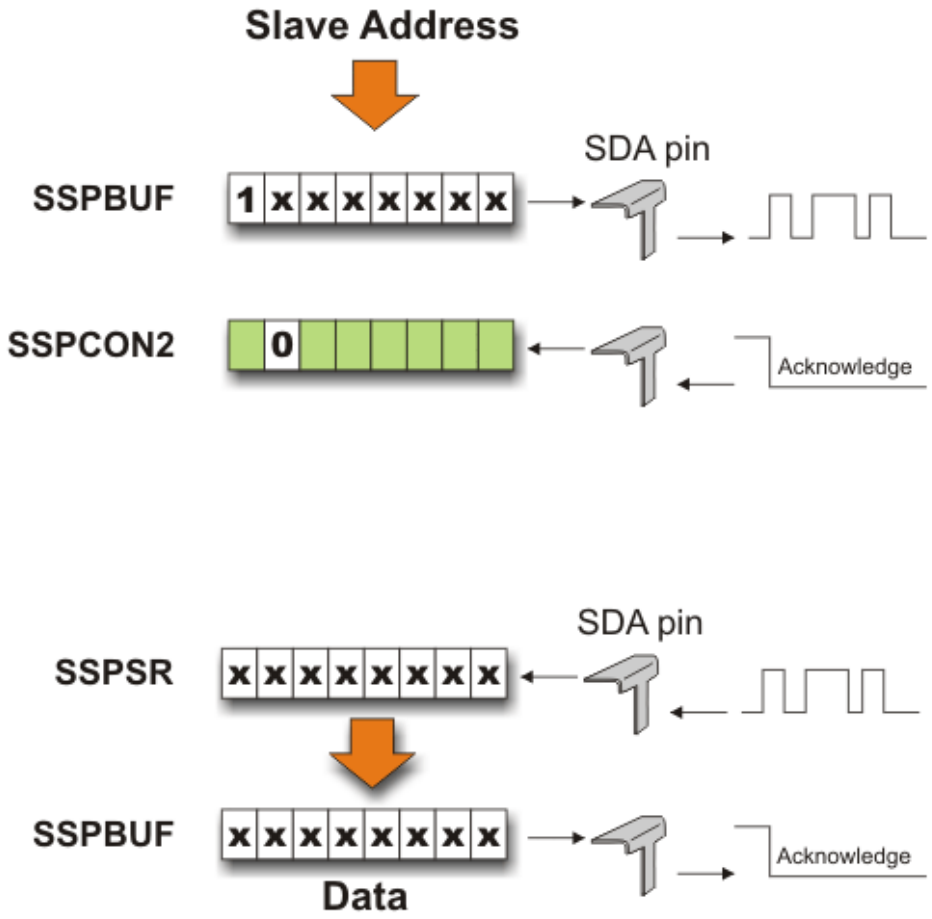
Data Transmission in I2C Master Mode

- Truyền dữ liệu trên chân **SDA** bắt đầu bằng mức logic 0 khi set bit **SEN** trong thanh ghi **SSPCON2**.
- Sau khi bật, vi điều khiển phải chờ một khoảng thời gian để thực hiện **Start condition** (chuẩn bị nội bộ và kiểm tra).
- Nếu điều kiện hợp lệ, bit **SSPIF** trong **PIR1** được set → báo sẵn sàng truyền dữ liệu.
- Truyền dữ liệu bắt đầu khi ghi byte vào **SSPBUF**.
- **Quy trình truyền I²C Master:**
- **Start condition**: SEN = 1 → SDA xuống mức thấp.
- **Địa chỉ Slave (7-bit)** + bit thứ 8 (R/W):
  - 0 = ghi dữ liệu (Master → Slave).
  - 1 = đọc dữ liệu (Slave → Master).
- Tất cả Slave nhận byte đầu tiên, nhưng chỉ Slave có địa chỉ khớp phản hồi.
- **ACK**: Slave xác nhận bằng cách xóa bit **ACKSTAT** trong **SSPCON2**.
- Nếu ACK hợp lệ → truyền dữ liệu tiếp tục: **Data** → **ACK** → **Data** → **ACK...**
- Kết thúc truyền bằng **Stop condition**: set bit **PEN** trong **SSPCON2** → SDA trở lại mức cao.

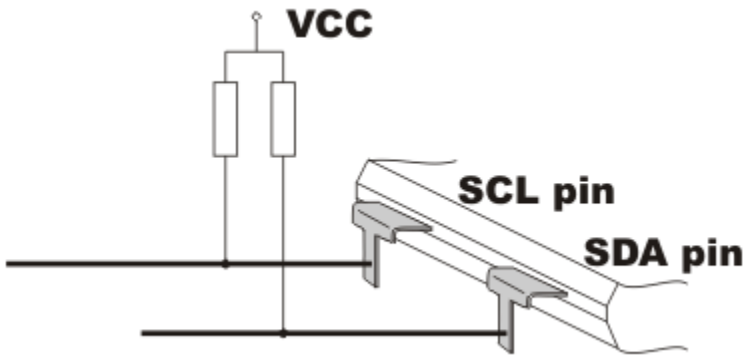
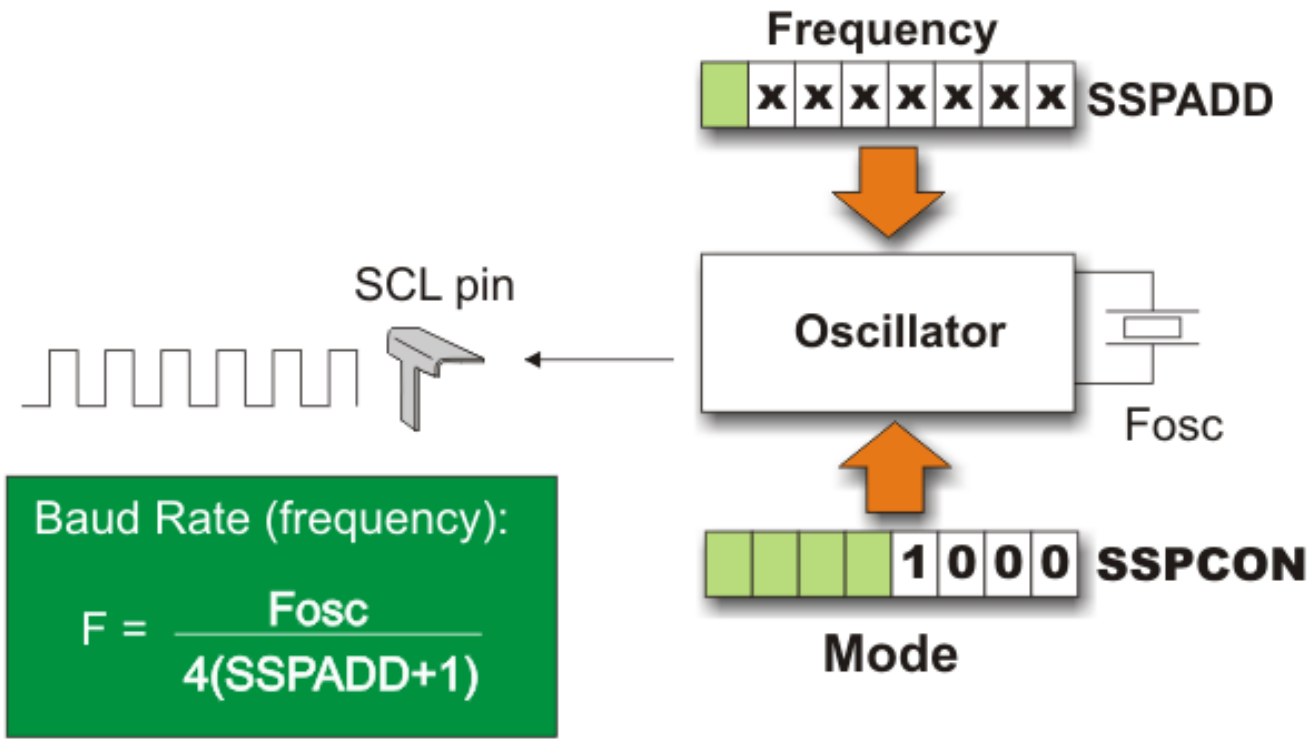


Data Reception in I²C Master Mode

- Chuẩn bị nhận dữ liệu gần giống như truyền dữ liệu, ngoại trừ bit cuối cùng của byte đầu tiên (địa chỉ + R/W) được set thành 1 → báo rằng Master muốn **nhận dữ liệu** từ Slave.
- Sau khi hoàn tất chuẩn bị và **Start condition** được khởi tạo, Slave bắt đầu gửi dữ liệu từng byte.
- Mỗi byte được dịch tuần tự vào thanh ghi **SSPSR**.
- Khi nhận đủ 8 bit, dữ liệu được tự động chuyển sang **SSPBUF** để Master đọc.
- Khi Master đọc SSPBUF → một bit **ACK** được gửi tự động đến Slave, báo rằng Master sẵn sàng nhận byte tiếp theo.
- Quá trình tiếp tục: **Data → ACK → Data → ACK...** cho đến khi Master khởi tạo **Stop condition** bằng cách set bit **PEN**.



Chuỗi xung trong quá trình nhận:

$$Start \rightarrow Address + R/W = 1 \rightarrow ACK \rightarrow Data \rightarrow ACK \ldots \rightarrow Stop$$

- Tất cả sự kiện trên chân **SDA** phải được đồng bộ với clock từ Master.
- Clock này được tạo bởi bộ dao động nội, tần số phụ thuộc vào:
  - Tần số thạch anh chính của vi điều khiển.
  - Giá trị trong thanh ghi **SSPADD**.
  - Chế độ SPI/I²C hiện tại.